arXiv:2206.00794v2 [stat.ML] 19 Aug 2024

Sequential Bayesian Neural Subnetwork Ensembles

Sanket Jantre[1]*, Shrijita Bhattacharya[2], Nathan M. Urban[1], Byung-Jun Yoon[1,3], Tapabrata Maiti[2], Prasanna Balaprakash[4], Sandeep Madireddy[5]*

[1] Computational Science Initiative, Brookhaven National Laboratory
[2] Department of Statistics and Probability, Michigan State University
[3] Department of Electrical and Computer Engineering, Texas A&M University
[4] Computing and Computational Sciences Directorate, Oak Ridge National Laboratory
[5] Mathematics and Computer Science Division, Argonne National laboratory
{sjantre, nurban, byoon}@bnl.gov, {bhatta61, maiti}@msu.edu, pbalapra@ornl.gov, smadireddy@anl.gov

Abstract

Deep ensembles have emerged as a powerful technique for improving predictive performance and enhancing model robustness across various applications by leveraging model diversity. However, traditional deep ensemble methods are often computationally expensive and rely on deterministic models, which may limit their flexibility. Additionally, while sparse subnetworks of dense models have shown promise in matching the performance of their dense counterparts and even enhancing robustness, existing methods for inducing sparsity typically incur training costs comparable to those of training a single dense model, as they either gradually prune the network during training or apply thresholding post-training. In light of these challenges, we propose an approach for sequential ensembling of dynamic Bayesian neural subnetworks that consistently maintains reduced model complexity throughout the training process while generating diverse ensembles in a single forward pass. Our approach involves an initial exploration phase to identify high-performing regions within the parameter space, followed by multiple exploitation phases that take advantage of the compactness of the sparse model. These exploitation phases quickly converge to different minima in the energy landscape, corresponding to high-performing subnetworks that together form a diverse and robust ensemble. We empirically demonstrate that our proposed approach outperforms traditional dense and sparse deterministic and Bayesian ensemble models in terms of prediction accuracy, uncertainty estimation, out-of-distribution detection, and adversarial robustness.

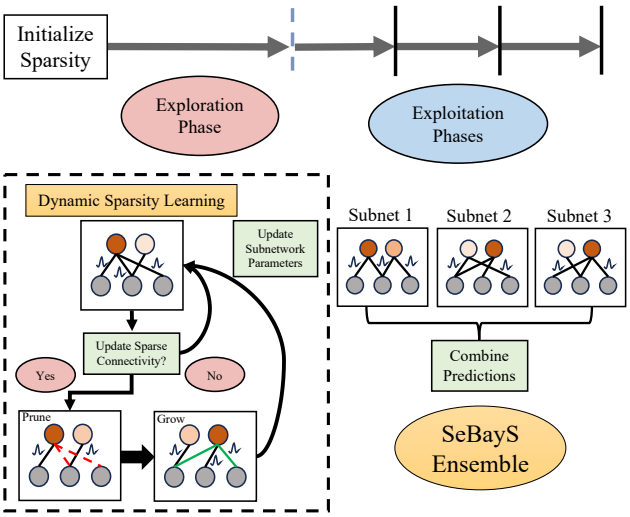

Figure 1: Illustration of `SeBayS` ensemble: Our approach includes an *exploration phase* followed by multiple *exploitation phases* to create Bayesian subnetworks. `SeBayS` ensemble prediction is obtained by combining their predictions.

1 Introduction

Deep learning has powere state-of-the-art performance in a wide array of machine learning tasks (LeCun, Bengio, and Hinton 2015). However, deep learning models still face many fundamental issues from the perspective of statistical modeling, which are crucial in many fields including autonomous driving, healthcare, and science (Bartlett, Montanari, and Rakhlin 2021). One of the major challenges is their ability to reliably estimate model uncertainty while capturing complex data dependencies and being computationally tractable. Probabilistic machine learning, especially the Bayesian framework, offers an exciting avenue to address these challenges. Besides superior uncertainty quantification, Bayesian models exhibit improved robustness to noise and adversarial perturbations (Wicker et al. 2021) due to probabilistic prediction capabilities. Bayesian neural networks (BNNs) have pushed the envelope of probabilistic machine learning through the combination of deep neural network (DNN) architecture and Bayesian inference. However, due to the enormous number of parameters, BNNs adopt approximate inference techniques such as variational inference with a fully factorized approximating family (Jordan et al. 1999). Although this approximation is crucial for computational tractability, it may under-utilize BNN's true potential (Izmailov et al. 2021).

Ensembles of neural networks (Lakshminarayanan, Pritzel, and Blundell 2017) have been proposed to account for the parameter/model uncertainty, which are shown to be analogous to Bayesian model averaging and sampling from the parameter posteriors to estimate the posterior predictive distribution (Wilson and Izmailov 2020). In this spirit, ensemble diversity is a key to enhancing predictions, uncertainty, and robustness. To this end, diverse ensembles can mitigate shortcomings of approximate Bayesian inference without sacrificing computational efficiency. In past, various diversity-inducing tech-

*Corresponding authors

niques have been explored, including using specific learning rate schedule (Huang et al. 2017), kernalized repulsion terms in the loss function (D'Angelo and Fortuin 2021), mixtures of approximate posteriors capturing multiple posterior modes (Dusenberry et al. 2020), sparsity as a mechanism for diversity (Havasi et al. 2021; Liu et al. 2022), and diversity in model architectures via neural architecture and hyperparameter searches (Egele et al. 2022; Wenzel et al. 2020).

However, most approaches use parallel ensembles, where individual model part starts with a different initialization. This can be computationally expensive, as each ensemble member requires extended training to reach a high-performing neighborhood of the parameter space. Although ensemble diversity is emphasized, the training cost is often overlooked. With the size of models only growing as we advance in deep learning, reducing the training cost of ensemble models alongside increasing their diversity is crucial.

Sequential ensembling offers an elegant solution to reduce the cost of obtaining multiple ensembles, with roots in methods that combine learning trajectory epochs (Swann and Allinson 1998; Xie, Xu, and Chuang 2013). (Jean et al. 2015; Sennrich, Haddow, and Birch 2016) leverage intermediate training stages, while (Moghimi et al. 2016) uses boosting to generate ensembles. Recent methods (Huang et al. 2017; Garipov et al. 2018; Liu et al. 2022) employ cyclic learning rate annealing to force the model to visit multiple local minimas, collecting ensembles at each local minimum. All these techniques have been primarily applied to deterministic models. Extending sequential ensembling to Bayesian models is attractive as it can allow creation of high-performing ensembles without the need to train from scratch, similar to sampling a posterior distribution with a Markov chain Monte Carlo sampler. Sequential ensembling can also complement parallel ensembling, where each parallel ensemble can generate multiple sequential ensembles, increasing the overall diversity of the final ensemble model.

A new frontier for improving the computational tractability and robustness of neural networks is *sparsity* (Hoefler et al. 2021). Famously, the lottery ticket hypothesis (Frankle and Carbin 2019) established the existence of sparse subnetworks that can match the performance of the dense model. Studies have also shown that such subnetworks tend to be inherently diverse due to different neural connectivity (Havasi et al. 2021; Liu et al. 2022; Yin et al. 2023). However, most sparsity-inducing techniques have focused on deterministic networks, using post-hoc pruning (Han, Mao, and Dally 2016; Molchanov, Ashukha, and Vetrov 2017). In Bayesian learning, the prior distribution provides a systematic approach to incorporate inductive bias and expert knowledge directly into the model (Robert et al. 2007). Consequently, sparsity in Bayesian neural networks can be introduced via sparsity-inducing priors (Louizos, Ullrich, and Welling 2017; Bai, Song, and Cheng 2020; Ghosh, Yao, and Doshi-Velez 2019; Jantre, Bhattacharya, and Maiti 2023a,b) which either incorporate sparsity gradually during training or post-training using thresholding criteria. These approaches successfully reduce computational and memory costs during inference, but their training costs remain similar to those of dense models.

To this end, we propose **Se**quential **Bay**esian Neural **S**ubnetwork Ensembles (`SeBayS`) with the following key contributions:

- We propose a sequential ensembling strategy for Bayesian neural networks (BNNs) which learns multiple subnetworks in a single forward-pass utilizing a fully sparse Bayesian framework that embeds sparsity in the posterior from the start of training. The approach involves a single exploration phase to find high-performing sparse network connectivity followed by multiple exploitation phases to obtain multiple subnetworks for ensembling.
- We leverage the light-weight dynamic sparsity learning to efficiently generate diverse sparse Bayesian neural networks, which we refer to as *Bayesian neural subnetworks*. Our approach outperforms current state-of-the-art methods in terms of predictive accuracy, uncertainty estimation, out-of-distribution detection, and adversarial robustness.

2 Related Work

Ensembles of neural networks: Ensembling techniques in the context of neural networks are increasingly being adopted in the literature due to their potential to improve accuracy, robustness, and quantify uncertainty. The most simple and widely used approach is Monte Carlo dropout, which is based on Bernoulli noise (Gal and Ghahramani 2016) and deactivates certain units during training and testing. This, along with techniques such as DropConnect (Wan et al. 2013), Swapout (Singh, Hoiem, and Forsyth 2016) are referred to as"implicit" ensembles as model ensembling is happening internally in a single model. Although they are efficient, the gain in accuracy and robustness is limited, and they are mainly used in the context of deterministic models. Although most recent approaches have targeted parallel ensembling techniques, few approaches such as BatchEnsemble (Wen, Tran, and Ba 2020) appealed to parameter efficiency by decomposing ensemble members into a product of a shared matrix and a rank-one matrix, and using the latter for ensembling and MIMO (Havasi et al. 2021) which discovers subnetworks from a larger network via multi-input multi-output configuration. In the context of Bayesian neural network ensembles, (Dusenberry et al. 2020) proposed a rank-1 parameterization of BNNs, where each weight matrix involves only a distribution on a rank-1 subspace and uses mixture approximate posteriors to capture multiple modes. (Premchandar et al. 2022) employ weight space ensembling with distribution learning of architectures for improved robustness.

Sequential ensembling techniques offer an elegant solution to ensemble training but have not received much attention recently due to a wider focus of the community on diversity of ensembles and less on the computational cost. Notable sequential ensembling techniques are (Huang et al. 2017; Garipov et al. 2018; Liu et al. 2022) that enable the model to visit multiple local minima through cyclic learning rate annealing and collect ensembles only when the model reaches a local minimum. The difference is that (Huang et al. 2017) adopts cyclic cosine annealing, (Garipov et al. 2018) uses a piece-wise linear cyclic learning rate schedule that is inspired by geometric insights. Finally, (Liu et al. 2022) adopts a piece-wise constant cyclic learning rate schedule. We also

note that all of these approaches have been primarily in the context of deterministic neural networks.

Our approach (i) introduces sequential ensembling into Bayesian neural networks, (ii) combines it with dynamic sparsity learning for cheaply collecting Bayesian subnetworks, and (iii) efficiently produces diverse model ensembles. It complements other parallel and efficient ensemble methods.

3 Sequential Bayesian Neural Subnetwork Ensembles

3.1 Preliminaries

Bayesian Neural Networks. Let $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1,\cdots,N}$ represent a training dataset of N i.i.d. observations, where $\boldsymbol{x}$ denotes input samples and y denotes corresponding outputs. In the Bayesian framework, instead of optimizing over a single probabilistic model, $p(y|\boldsymbol{x}, \boldsymbol{\theta})$, we discover all likely models via posterior inference over model parameters, $\boldsymbol{\theta} = (\theta_1, \cdots, \theta_T) \in \mathbb{R}^T$. The Bayes' rule provides the posterior distribution: $p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})$, where $p(\mathcal{D}|\boldsymbol{\theta})$ denotes the likelihood of $\mathcal{D}$ given $\boldsymbol{\theta}$ and $p(\boldsymbol{\theta})$ is the prior over parameters. Using $p(\boldsymbol{\theta}|\mathcal{D})$, we predict the label for a new example $\boldsymbol{x}^*$ through Bayesian model averaging:

$$p(y^*|\boldsymbol{x}^*, \mathcal{D}) = \int p(y^*|\boldsymbol{x}^*, \boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta}$$
$$\approx \frac{1}{M}\sum_{m=1}^{M} p(y^*|\boldsymbol{x}^*, \boldsymbol{\theta}_m), \quad \boldsymbol{\theta}_m \sim p(\boldsymbol{\theta}|\mathcal{D})$$

Variational Inference (VI). Although, Markov chain Monte Carlo sampling is the gold standard for Bayesian inference, it is computationally inefficient (Izmailov et al. 2021). Variational inference, in contrast, is faster and scales well for complex tasks with large datasets (Blei, Kucukelbir, and McAuliffe 2017). Variational learning infers a distribution $q(\boldsymbol{\theta})$ on model parameters that minimises the Kullback-Leibler (KL) distance from the true posterior $p(\boldsymbol{\theta}|\mathcal{D})$:

$$\widehat{q}(\boldsymbol{\theta}) = \underset{q(\boldsymbol{\theta})\in\mathcal{Q}}{\text{argmin}}\ d_{\text{KL}}(q(\boldsymbol{\theta}), p(\boldsymbol{\theta}|\mathcal{D}))$$

where $\mathcal{Q}$ denotes the variational family of distributions. This optimization problem is equivalent to minimizing the negative Evidence Lower Bound (ELBO), defined as

$$\mathcal{L} = -\mathbb{E}_{q(\boldsymbol{\theta})}[\log p(\mathcal{D}|\boldsymbol{\theta})] + d_{\text{KL}}(q(\boldsymbol{\theta}), p(\boldsymbol{\theta})), \qquad (1)$$

where the first term is the data-dependent cost, known as the negative log-likelihood (NLL), and the second term is prior-dependent and serves as regularization. The NLL is often intractable and estimated using Monte Carlo sampling. Direct optimization of (1) is computationally prohibitive, so gradient descent methods are employed (Kingma and Welling 2014), with the reparameterization trick used for efficient gradient backpropagation.

Prior Choice. Dynamic sparsity learning to obtain Bayesian subnetworks is achieved by randomly selecting a sparse connectivity of preferred dimension s. Thus, a prior on $\boldsymbol{\theta}$ that incorporates this sparsity can be defined as follows:

$$p(\theta_i) = \gamma_i \mathcal{N}(0, \sigma_0^2), \ \ \gamma_i = \mathbf{1}_S(i) \text{ with } S = \{n_1, \cdots, n_s\}$$

Here, $\mathbf{1}_S(.)$ is the indicator function on the set S of non-zero weight indices. The variational family mirrors the prior's structure to maintain sparsity in variational approximation. The mean-field variational family structure is:

$$q_{\phi_i}(\theta_i) = \gamma_i \mathcal{N}(\mu_i, \sigma_i^2), \ \ \gamma_i = \mathbf{1}_S(i), \ S = \{n_1, \cdots, n_s\}$$

here, $\phi_i = (\mu_i, \sigma_i^2)$ represents the variational mean and variance parameters of $q_{\phi_i}(\theta_i)$. For sparse learning, $(\mu_i, \sigma_i^2) = (0, 0)$ when $i \notin \{n_1, \cdots, n_s\}$. We derive the ELBO from (1) to formulate the optimization problem as minimizing

$$\mathcal{L} = -\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})}[\log p(\mathcal{D}|\boldsymbol{\theta})] + \sum_{i=1}^{T} \gamma_i d_{\text{KL}}(q_{\phi_i}(\theta_i), p(\theta_i)) \quad (2)$$

This formulation maintains the target sparsity level s throughout model training, reducing computational and memory demands. Periodically, we explore the parameter space to enhance sparse connectivity by pruning a fraction p of the least significant weights from $\boldsymbol{\theta}_s$, followed by regrowing an equivalent number of weights during training. This approach improves upon static sparsity methods while preserving the same computational budget.

3.2 Bayesian Neural Subnetworks

We follow the key steps of the deterministic dynamic sparsity learning method, Rigged Lottery (Evci et al. 2020): initial sparse connectivity, model weight optimization, and the prune-grow criterion to collect Bayesian subnetworks.

Initial Sparse Connectivity. We initialize the sparsity with *Erdős-Rényi-Kernel* (ERK) method, which scales the sparsity of the lth-convolution layer by a factor of $1 - \frac{n^{l-1}+n^l+w^l+h^l}{n^{l-1}\times n^l \times w^l \times h^l}$, where w^l and h^l are the width and height of the lth-layer convolution kernel and n^l are the number of channels in the lth- convolution layer. For the lth-linear layer, the sparsity scales with a factor of $1 - \frac{n^{l-1}+n^l}{n^{l-1}\times n^l}$. This sparse allocation ensures that layers with more parameters are pruned more aggressively. Once the initial set of sparse indices S_{init} is selected, we initialize the $\boldsymbol{\phi}_0$ on S_{init} following the procedure used in VI-based BNN techniques.

Model Weight Optimization. For a given $\boldsymbol{\gamma}$, $\boldsymbol{\phi}$ is optimised using the standard VI-based BNN training approach with the SGD algorithm for ΔT steps as outlined in (Li et al. 2024). During the forward pass through the sparse BNN, the NLL term in (2) is estimated via Monte Carlo sampling from $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$, employing local-reparameterization trick for efficiency (Kingma, Salimans, and Welling 2015). After ΔT steps, Lastly, upon updating the sparse connectivity ($\boldsymbol{\gamma}^{\tau} \rightarrow \boldsymbol{\gamma}^{\tau+1}$), the newly included non-zero weights (denoted by indices j) initially have zero $\sigma_j^{\tau+1}$ values that impact the gradient descent since their gradients are also zero. We set $\sigma_j^{\tau+1}$s to the average of the non-zero σ_k^{τ} values from the same convolution kernel or fully connected layer as $\sigma_j^{\tau+1}$.

Prune-Grow Criterion. We update $\boldsymbol{\phi}$ and $\boldsymbol{\gamma}$ using an alternating optimization strategy. After every ΔT steps, the sparse connectivity $\boldsymbol{\gamma}$ is updated deterministically as described in (Li et al. 2024). Specifically, $\boldsymbol{\gamma}^{\tau+1}$ is obtained by pruning a